MATTER TEST HARNESS · USER GUIDE

TC-CNET-4.12

Validating ConnectNetwork on Thread with two nRF OpenThread Border Routers

Raspberry Pi OTBR (network 1) + TH otbr-chip (network 2)

CLUSTER	PICS	RUN TIME	RADIOS	AUTOMATION
Network Commissioning	CNET.S.F01	~3 min	2 × nRF52840 RCP	cnet412_setup.sh

In short

TC-CNET-4.12 moves the DUT between two Thread networks and checks that the TH can always find it again. So you need **two Thread border routers** on the same LAN: a separate Raspberry Pi (the **Thread Pi**) for network 1, and the **TH** itself for network 2.

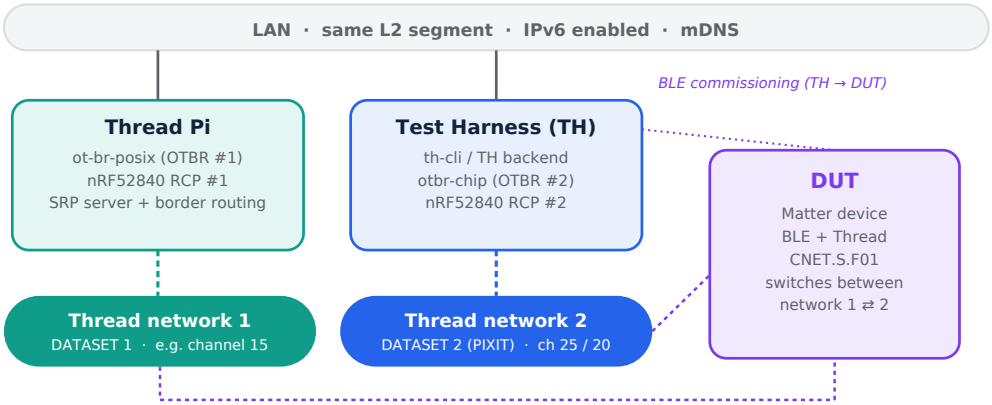
Part	What	Who
1	Common setup: hardware and software, done once	Everyone
2	Automated method: the cnet412_setup.sh script	QA
3	Manual method: every step by hand	Anyone without the script

Based on src/python_testing/TC_CNET_4_12.py (connectedhomeip master) and certification-tool PR #1181.
Verify commands against your SDK / TH versions. Datasets contain network keys: keep them private.

Before you start

Two names are used throughout this guide. Please read them first.

Name	Means	Runs
Thread Pi	The separate Raspberry Pi that provides Thread network 1. It is NOT the TH.	ot-br-posix + nRF52840 radio #1
TH	The Matter Test Harness (may itself be a Raspberry Pi). It provides Thread network 2.	cert tool, th-cli, otbr-chip + radio #2



The DUT starts on network 1 and is moved to network 2, back to 1, to 2 and back to 1. After each move the TH finds it again by mDNS. This only works if the TH can reach **both** networks over the LAN.

Equipment

What	Notes
2 × nRF52840 dongle or DK	Same firmware on both: one in the Thread Pi, one in the TH.
Thread Pi	Raspberry Pi 4 with Ubuntu or Raspberry Pi OS.
TH	Cert tool version that includes scripts/utils/run_cnet_4_12.sh.
DUT	BLE + Thread. PICS CNET.S.F01 = 1.
LAN	Thread Pi and TH on the same network. Ethernet is best; Wi-Fi works if devices can reach each other.

Part 1 · Common setup (everyone, once)

Needed for both methods. Done once per bench; it survives reboots.

Step	What	Where
C1	Flash both radios	PC
C2	Install the border router on the Thread Pi	Thread Pi
C3	Configure and start it	Thread Pi
C4	Check the TH	TH
C5	Prepare the DUT	DUT

C1 Flash both radios

WHERE: Your PC

Both nRF52840 boards get the OpenThread RCP (radio co-processor) firmware.

```
# in your nRF Connect SDK workspace
west build -b nrf52840dongle/nrf52840 samples/openthread/coprocessor
west flash          # DK;  dongle: nrfutil DFU (see the NCS docs for your version)
```

EXPECTED RESULT

Plug one board into the Thread Pi and one into the TH. On each, `ls /dev/ttyACM*` shows `/dev/ttyACM0`.

C2 Install the border router

WHERE: Thread Pi

Installs ot-br-posix on the Thread Pi. Use eth0 instead of wlan0 if the Pi is wired.

On the Thread Pi

```
git clone --depth=1 https://github.com/openthread/ot-br-posix
cd ot-br-posix
./script/bootstrap
INFRA_IF_NAME=wlan0 ./script/setup
```

EXPECTED RESULT

The setup finishes without errors (several minutes).

C3 Configure and start the border router

WHERE: Thread Pi

Tells it which radio and which network interface to use.

Run `sudo nano /etc/default/otbr-agent` and make the file contain exactly:

/etc/default/otbr-agent on the Thread Pi

```
OTBR_AGENT_OPTS="-I wpan0 -B wlan0 spinel+hdlc+uart:///dev/ttyACM0 trel://wlan0"
OTBR_NO_AUTO_ATTACH=0
```

```
sudo systemctl restart otbr-agent
sudo systemctl is-active otbr-agent       # prints: active
```

IMPORTANT **If not**

Check the radio (`ls /dev/ttyACM*`) and the log: `sudo journalctl -u otbr-agent -n 50`.

C4 Check the TH

WHERE: Test Harness

The cert tool must contain the PR #1181 script, and radio #2 must be plugged in.

On the TH

```
ls ~/certification-tool/scripts/utils/run_cnet_4_12.sh
th-cli available-tests | grep TC_CNET_4_12
ls /dev/ttyACM0
```

EXPECTED RESULT

All three commands print a result (no 'No such file').

C5 Prepare the DUT

WHERE: DUT

Note the onboarding values and set the PICS.

Write down the DUT's **discriminator** and **setup code**, and set `CNET.S.F01=1` in the TH project PICS. Factory-reset the DUT only right before a test run (BLE advertising times out).

Part 2 · Automated method (QA)

One script on the TH does all of Part 3 for you. It asks for the values it needs and remembers them.

A1 One-time: script and SSH key

WHERE: Test Harness

The script configures the Thread Pi over SSH, so the TH needs an SSH key for it.

On the TH

```
chmod +x ~/cnet412_setup.sh
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519      # only if you have no SSH key yet
ssh-copy-id ubuntu@<Thread Pi IP>
ssh ubuntu@<Thread Pi IP> true && echo OK             # must print OK without a password
```

A2 Prepare: ./cnet412_setup.sh setup

WHERE: Test Harness

Once per session and after every reboot of the TH or the Thread Pi. Takes about 2 minutes.

Example: first run. Next time every question shows [Press Enter to reuse: ...].

```
$ ./cnet412_setup.sh setup
==> Questions
Thread Pi = the Raspberry Pi running the border router for Thread network 1 (not the TH).
  Thread Pi IP address: 192.168.0.215
  Thread Pi SSH user [Press Enter for default: ubuntu]:
  DUT discriminator (0-4095): 3840
  DUT setup code (8 digits): 20202021
  TH network interface towards the Thread Pi (eth0/wlan0) [Press Enter for default: wlan0]:
==> Thread Pi (ubuntu@192.168.0.215): ...   (asks for the Pi's sudo password)
...
Setup done. Next: ./cnet412_setup.sh run
```

The script does	Same as manual step
Creates network 1 on the Thread Pi (or keeps it), enables SRP and forwarding, opens the firewall	M1, M2
Applies the two cert tool fixes and the TH network settings	M3, M4
Creates network 2 and the test config, removes old prefixes, checks the route	M5
Checks everything	M6

A3 Run the test: ./cnet412_setup.sh run

WHERE: TH + DUT

Every run. It checks first, shows the DUT values from the last setup, then asks you to reset the DUT. Testing a different DUT? Run setup again first.

```
./cnet412_setup.sh run
# factory-reset the DUT when asked, press Enter, then answer:
#   "reuse the previous commissioning information?"  -> 2 (NO)
#   "Make sure the DUT is in Commissioning Mode"     -> 1 (DONE)
```

EXPECTED RESULT

`Test Case Completed [PASSED]: TC-CNET-4.12` after about 4 minutes. Factory-reset the DUT before the next run.

Other commands

Command	Use it when
./cnet412_setup.sh check	You want to know if everything is ready, without running the test
./cnet412_setup.sh diag	A run failed: saves the state of both border routers to a file for the report

To change a remembered value, edit or delete `~/.cnet412.conf`. Environment variables (THREAD_PI_IP, DUT_DISCRIMINATOR, ...) override it.

Part 3 · Manual method

Steps M1–M4 are done once. Steps M5–M7 are done for every test run.

Worksheet: fill in as you go

	Value	From	Example
W1	Thread Pi IP address	your network	192.168.0.215
W2	Network interface (Thread Pi and TH)	ip -br link	wlan0 or eth0
W3	DATASET 1 (long hex, keep private)	M1	0e080000...0f
W4	Network 1 prefix	M1	fdbb:9c0f:d617:1::/64
W5	Thread Pi link-local address	M1	fe80::de77:f429:1772:1c97
W6	DUT discriminator / setup code	C5	3840 / 20202021

M1 Create Thread network 1 (once)

WHERE: Thread Pi

Creates network 1 on channel 15 and reads W3–W5.

1. Create (on the Thread Pi)

```
sudo ot-ctl thread stop
sudo ot-ctl dataset init new
sudo ot-ctl dataset channel 15
sudo ot-ctl dataset commit active
sudo ot-ctl ifconfig up
sudo ot-ctl thread start
```

2. Check and fill in the worksheet

```
sudo ot-ctl state                  # wait until: leader  (10-20 s)
sudo ot-ctl srp server state       # must be: running
sudo ot-ctl dataset active -x      # -> W3
sudo ot-ctl br omrprefix           # "Local:" prefix -> W4
ip -6 addr show wlan0 scope link   # fe80:: address -> W5
```

IMPORTANT **If not**

SRP not running: `sudo ot-ctl srp server enable`. Not leader after a minute: restart otbr-agent (C3).

M2 Open the firewall (once)

WHERE: Thread Pi

If Docker is installed on the Thread Pi, it blocks traffic coming out of the Thread network.

Check (on the Thread Pi)

```
sudo ip6tables -S FORWARD | head -1      # "-P FORWARD ACCEPT" -> skip to M3
sysctl net.ipv6.conf.all.forwarding      # must be 1
```

Only if the first line shows `-P FORWARD DROP`, add this permanent rule:

Fix (only if DROP)

```
sudo tee /etc/systemd/system/otbr-docker-fwd.service > /dev/null <<'EOF'
[Unit]
Description=Allow Thread -> LAN IPv6 forwarding with Docker
After=docker.service otbr-agent.service

[Service]
Type=oneshot
ExecStart=/bin/sh -c 'ip6tables -C DOCKER-USER -i wpan0 -o wlan0 -j ACCEPT 2>/dev/null \\
    || ip6tables -I DOCKER-USER -i wpan0 -o wlan0 -j ACCEPT'
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now otbr-docker-fwd.service
```

EXPECTED RESULT

`sudo ip6tables -S DOCKER-USER` lists `-i wpan0 -o wlan0 -j ACCEPT`.

M3 Two cert tool fixes (once)

WHERE: Test Harness

Two known issues in the TH scripts.

Fix 1: avahi path (on the TH)

```
cd ~/certification-tool
F=backend/test_collections/matter/scripts/OTBR/otbr_start.sh
OLD=backend/app/otbr_manager/avahi
NEW=backend/test_collections/matter/sdk_tests/support/otbr_manager/avahi
sed -i "s#$OLD#$NEW#" "$F"
```

Fix 2: in `scripts/utils/run_cnet_4_12.sh`, find the line with `SRP server is not running` (about line 204) and put this line directly **above** it:

Fix 2: add above the SRP check (4 spaces at the start)

```
for _ in $(seq 1 30); do [[ "$(ot_out srp server state)" == "running" ]] && break; sleep 1; done
```

EXPECTED RESULT

`grep -c 'seq 1 30' scripts/utils/run_cnet_4_12.sh` prints 1.

M4 Route to network 1 (once)

WHERE: Test Harness

The TH must learn the route to network 1 from the Thread Pi.

On the TH (replace wlan0 with W2)

```
printf 'net.ipv6.conf.wlan0.accept_ra=2\nnet.ipv6.conf.wlan0.accept_ra_rt_info_max_plen=64\n' \
    | sudo tee /etc/sysctl.d/99-thread-ra.conf
sudo sysctl --system > /dev/null
sleep 60
ip -6 route get <W4 with ::1 instead of ::/64>     # e.g. fdbb:9c0f:d617:1::1
```

EXPECTED RESULT

The last command shows `via [W5] dev wlan0`.

IMPORTANT If not
Add it by hand (repeat after a TH reboot): `sudo ip -6 route replace [W4] via [W5] dev wlan0`.

M5 Network 2 and test config (every run)

WHERE: Test Harness

Starts the TH's border router (otbr-chip), creates network 2 and writes ~/cnet412_config.json.

On the TH

```
cd ~/certification-tool
export DUT_DISCRIMINATOR=3840 DUT_SETUP_CODE=20202021     # W6
export THREAD1_DATASET_HEX=<W3>
export INFRA_IFACE=wlan0                                   # W2
scripts/utils/run_cnet_4_12.sh --start-otbr --prepare-only
```

EXPECTED RESULT
The output ends with `network 2: … prefix fd11:xxxx::/64` and `Prepared.`

Then remove network 2 prefixes left from earlier runs (keep only the fd11:xxxx prefix just printed):

```
docker exec otbr-chip ot-ctl prefix                          # list
docker exec otbr-chip ot-ctl prefix remove fd11:35:0:0::/64  # each OLD fd11 prefix
docker exec otbr-chip ot-ctl netdata register
```

M6 Check everything (every run)

WHERE: Test Harness

All six must show the expected value.

#	Check	Command	Expected
1	Network 2 up	`docker exec otbr-chip ot-ctl state`	leader
2	Network 2 SRP	`docker exec otbr-chip ot-ctl srp server state`	running
3	Network 1 up	`ssh -t @[W1] sudo ot-ctl state`	leader
4	Route to network 1	`ip -6 route get`	via [W5]
5	One network 2 prefix	`docker exec otbr-chip ot-ctl prefix`	one fd11 line
6	Runner free	`th-cli test-runner-status`	IDLE

M7 Run the test (every run)

WHERE: TH + DUT

Factory-reset the DUT, then start the test right away.

```
th-cli run-tests --tests-list TC_CNET_4_12 --config ~/cnet412_config.json
#   "reuse the previous commissioning information?"  -> 2 (NO)    within 60 s
#   "Make sure the DUT is in Commissioning Mode"     -> 1 (DONE)  within 60 s
```

The screen may stay silent for up to 3 minutes while the DUT is commissioned. That's normal.

EXPECTED RESULT
`Test Case Completed [PASSED]: TC-CNET-4.12`. Logs: `~/output_logs/`. Factory-reset the DUT before the next run.

Troubleshooting

Find the message, then do what the right column says. With the script, the fix is usually: run ./cnet412_setup.sh setup again.

WARNING Known issue: step 7 fails with 'No IP addresses found in SRP record'

A problem in the test's mDNS lookup, not in the DUT: right after the DUT moves to network 2, the lookup can miss the address that was sent with the record. A fix is proposed upstream. Until your TH contains it, report this failure as a known test-script issue and attach the TH log. A run with a locally modified script is not a valid certification result. QA: `./cnet412_setup.sh` check warns if the TH's script doesn't have the fix yet.

Preparing (M5 / setup)

Message or symptom	What to do
SRP server is not running on otbr-chip	Fix 2 (M3) is missing. Apply it, then repeat M5.
OTBR state is 'child'	An old DUT is still on network 2. Factory-reset the DUT, repeat M5.
same extpanid	Re-create network 1 (M1), update W3, repeat M5.
test runner is not IDLE	th-cli abort-testing, then repeat.
cannot reach … over SSH (script)	Check the Thread Pi IP and step A1 (SSH key).

Commissioning (before test step 1)

Message or symptom	What to do
BLE discovery fails / Secure Pairing Failed	The DUT stopped advertising. Reset it and start the test immediately. Check W6.
Prompt timed out, everything CANCELLED	Answer the prompts within 60 seconds.
ThreadNetworkEnable, then 3 × 'CASESession timed out' (~2.5 min silent)	The TH can't reach the DUT on network 1: check M2 (firewall) and M4 (route), M6 checks 3 and 4.

During the test

Message or symptom	What to do
Step 2: Thread network not found	THREAD1_DATASET_HEX isn't the Thread Pi's current dataset. Re-read W3 (M1).
Step 7: No IP addresses found in SRP record	Known issue, see the red box at the top of this page.
Step 7 / 16: SRV … not found	Network 2 isn't publishing the DUT. Check M6 checks 1, 2, 5; repeat M5.
Step 11 / 21: DUT doesn't return to network 1	Check network 1 is still leader (M6 check 3) and the DUT is in range of the Thread Pi.

For a report, attach: the failing step, the TH log from ~/output_logs/, the DUT console log, and the file from ./cnet412_setup.sh diag.

Appendix · Why each step is needed

Background only.

Step	Why
M1 channel 15	Network 2 uses channel 25 (or 20). The networks must differ in channel, ExtPANID and key.
M2 firewall	Docker sets the IPv6 FORWARD policy to DROP. OTBR only allows traffic into Thread, so replies coming out are dropped.
M3 fix 1	otbr_start.sh mounts an avahi path that no longer exists.
M3 fix 2	run_cnet_4_12.sh checks the SRP server immediately after enabling it; it needs a few seconds to start.
M4	otbr-chip turns on IP forwarding on the TH; Linux then ignores route advertisements unless accept_ra=2.
M5 prefixes	Each prepare adds a new on-mesh prefix; old ones stay and give the DUT extra addresses.
M7 config	run_cnet_4_12.sh without --prepare-only would rebuild network 2 with a new dataset.

Optional: border router web UI on the Thread Pi

Install with `WEB_GUI=1 INFRA_IF_NAME=wlan0 ./script/setup` in C2, then `sudo systemctl edit otbr-web`, add the lines below and restart otbr-web. Open `http://[W1]:8080/` (no login: test benches only).

```
[Service]
Environment="OTBR_WEB_OPTS=-I wpan0 -a 0.0.0.0 -p 8080"
```

Based on TC_CNET_4_12.py (connectedhomeip master) and certification-tool scripts/utils/run_cnet_4_12.sh (v2.16-beta3+winter2026). NCS and ot-br-posix commands can differ slightly between versions.